

PepitoBuscaError

Memoria técnica completa

Proyecto final de Desarrollo de Aplicaciones Multiplataforma

(DAM) Autor: Héctor Mora **Curso académico:** 2025-2026

Tecnologías principales: Java 17, Spring Boot 3.5.14, Maven, Spring MVC, Thymeleaf, Spring Data JPA, Hibernate, MySQL 8, H2, HTML5, CSS3, Bootstrap Icons, Chart.js, Leaflet, OpenStreetMap, JUnit, Mockito y PowerShell.

Subtítulo: Plataforma web MVC de ciberseguridad defensiva para pequeñas y medianas empresas, orientada al registro de compañías, análisis de riesgo, OSINT pasivo, recomendaciones, visualización ejecutiva y geolocalización con consentimiento.

2. Índice

1. Portada
2. Índice
3. Resumen ejecutivo
4. Objetivo del proyecto
5. Arquitectura general
6. Tecnologías utilizadas y justificación
7. Estructura del repositorio
8. pom.xml
9. application.properties
10. Modelo de base de datos
11. Entidades Java
12. Repositorios
13. Servicios
14. Controladores
15. Dashboard y gráficos
16. Módulo de empresas
17. Módulo de análisis de riesgo
18. Motor de riesgo
19. Indicadores
20. Recomendaciones y plan de acción
21. Módulo OSINT
22. APIs externas
23. Módulo de geolocalización

- 24. Public HTTPS links
- 25. Scripts PowerShell
- 26. Templates Thymeleaf
- 27. CSS y diseño visual
- 28. Error handling
- 29. Tests
- 30. Seguridad y privacidad
- 31. Uso de IA durante el desarrollo
- 32. Limitaciones actuales
- 33. Mejoras futuras
- 34. Conclusión

3. Resumen ejecutivo

PepitoBuscaError es una aplicación web desarrollada con Spring Boot MVC que he planteado como una plataforma de ciberseguridad defensiva para pequeñas y medianas empresas. Su objetivo no es explotar sistemas ni sustituir a una auditoría profesional completa, sino ofrecer una primera visión estructurada de la exposición digital de una organización: empresas registradas, análisis de riesgo, indicadores técnicos, recomendaciones, comprobaciones OSINT pasivas, cuadros de mando y enlaces de geolocalización basados en consentimiento.

El problema que resuelve es muy habitual en una pyme: la información de seguridad suele estar dispersa, no siempre se documenta la evolución del riesgo y muchas decisiones técnicas se toman sin una vista clara de prioridades. Esta aplicación centraliza datos básicos de la empresa, permite registrar análisis, calcula una puntuación de riesgo, clasifica los resultados por nivel y genera acciones recomendadas. Además, el módulo OSINT aporta una revisión pasiva de dominios, DNS, correo, TLS, cabeceras web y exposición pública sin realizar ataques ni pruebas intrusivas.

La plataforma incluye estos módulos principales:

- Gestión de compañías: alta, listado, búsqueda, detalle, edición y borrado.
- Análisis de riesgo: selección de indicadores, cálculo de puntuación y clasificación por niveles.
- Indicadores: catálogo de señales de riesgo y consulta de indicadores almacenados.
- Recomendaciones: plan de acción priorizado a partir de los indicadores detectados.
- Dashboard: métricas, tarjetas KPI, gráficos y tendencias recientes.
- OSINT: análisis de dominio, comprobación de correo corporativo y creación de informes pasivos guardados.
- Geolocalización: registro de dispositivos propios, enlace privado por token y actualización de ubicación solo tras permiso del navegador.
- Documentación interna, scripts de ayuda y pruebas automatizadas.

El valor de la aplicación como plataforma de ciberseguridad está en que traduce señales técnicas en información comprensible para un entorno de negocio. Un responsable de una pyme puede ver qué empresas tienen más riesgo, qué indicadores aparecen con más frecuencia, qué recomendaciones están pendientes y qué activos públicos requieren revisión. Desde el punto de

vista académico, el proyecto demuestra uso real de arquitectura MVC, persistencia relacional, validación, servicios, repositorios, DTOs, plantillas Thymeleaf, pruebas y documentación técnica.

4. Objetivo del proyecto

El objetivo principal del proyecto es construir una aplicación web defendible, mantenible y coherente que permita analizar de forma básica la exposición digital de una empresa desde un enfoque defensivo. El proyecto se centra en explicar y organizar el riesgo, no en realizar explotación ofensiva.

En concreto, la aplicación persigue estos objetivos:

- Registrar compañías con su nombre, dominio, correo corporativo y sector.
- Crear análisis de riesgo asociados a cada compañía.
- Calcular una puntuación de riesgo a partir de indicadores definidos en el código.
- Asignar un nivel de riesgo comprensible: bajo, medio, alto o crítico.
- Generar recomendaciones concretas que convierten los hallazgos en acciones de mejora.
- Visualizar KPIs y gráficos para entender el estado global del sistema.
- Ejecutar comprobaciones OSINT pasivas sobre dominios y correos autorizados.
- Guardar informes OSINT con hallazgos, severidades, evidencias y recomendaciones.
- Gestionar dispositivos propios o autorizados mediante geolocalización consentida.
- Documentar configuración, modelo de datos, pruebas, privacidad y mejoras futuras.

Como estudiante, he diseñado el proyecto para que sirva tanto como demostración académica como prototipo profesional. La parte académica está en la aplicación de patrones MVC, JPA, validación, servicios y pruebas. La parte profesional está en la orientación a pymes, la separación de responsabilidades, la privacidad del módulo GPS, el uso de variables de entorno para secretos y el enfoque ético del OSINT.

5. Arquitectura general

La arquitectura general sigue el patrón Spring Boot MVC clásico. Esta decisión encaja muy bien con un proyecto final de DAM porque permite explicar de forma clara cómo entra una petición HTTP, cómo se procesa en Java, cómo se consulta o modifica la base de datos y cómo se renderiza finalmente una página HTML con Thymeleaf.

El flujo principal de la aplicación es:

```
Navegador
|
v
Controller Spring MVC
|
v
Service
|
v
Repository Spring Data JPA
|
v
Base de datos MySQL
|
v
Repository → Service → Controller
|
v
Vista Thymeleaf renderizada en HTML
```

La separación de capas que se observa en el código es la siguiente:

Capa	Paquetes / recursos	Responsabilidad
Presentación	<code>controller</code> , <code>templates</code> , <code>static</code>	Recibir rutas, validar formularios, preparar modelos y renderizar HTML.
Lógica de negocio	<code>service</code> y <code>service.checks</code>	Crear análisis, calcular riesgo, generar recomendaciones, ejecutar OSINT, gestionar geolocalización y enlaces públicos.
Persistencia	<code>repository</code>	Acceso a base de datos mediante Spring Data JPA.
Dominio	<code>model</code>	Entidades JPA, relaciones y enums de negocio.
Transporte / formularios	<code>dto</code>	Formularios y objetos de resultado que no son entidades persistentes directas.
Utilidades	<code>util</code>	Normalización de dominios y URLs.
Configuración	<code>application.properties</code> , <code>pom.xml</code>	Puerto, base de datos, claves API opcionales, dependencias y pruebas.

Esta arquitectura es apropiada porque evita mezclar responsabilidades. Los controladores no contienen SQL ni cálculo de riesgo; los repositorios no conocen la interfaz web; las entidades representan el modelo persistente; y los servicios concentran las decisiones de negocio. Para una defensa oral, esta estructura permite explicar el proyecto por capas y demostrar que cada clase tiene un propósito concreto.

6. Tecnologías utilizadas y justificación

Las tecnologías utilizadas aparecen directamente en el `pom.xml`, en las plantillas, en los recursos estáticos, en la configuración y en los tests.

Tecnología	Dónde aparece	Qué hace	Por qué encaja en el proyecto
Java 17	<code>pom.xml</code> con <code><java.version>17</java.version></code>	Lenguaje principal de la aplicación.	Es una LTS esta versión compatible con Spring Boot, adecuada para un proyecto DAM manteniéndose
Spring Boot 3.5.14	Parent Maven <code>spring-boot-starter-parent</code>	Arranque automático, servidor embebido, configuración y gestión de dependencias.	Reduce configuración repetitiva, permite centrarse en la lógica de aplicación
Maven	<code>pom.xml</code> , <code>mvnw</code> , <code>mvnw.cmd</code>	Compilación, dependencias, tests y ejecución.	Hace que el proyecto se reproduzca en Windows y otros entornos
Spring MVC	<code>spring-boot-starter-web</code> , controladores	Gestiona rutas HTTP, formularios y respuestas HTML.	El flujo de trabajo es claro y defendible, una aplicación server-side
Thymeleaf	<code>spring-boot-starter-thymeleaf</code> , <code>templates</code>	Motor de plantillas HTML del lado servidor.	Permite renderizar vistas sin necesidad de una SPA, API separada
Spring Data JPA	<code>spring-boot-starter-data-jpa</code> , repositorios	Genera operaciones	Reduce configuración repetitiva

Tecnología	Dónde aparece	Qué hace	Por qué encaja en el proyecto
		CRUD y consultas por nombre de método.	mantiene separada la persistencia
Hibernate	incluido por JPA	Implementación ORM que mapea entidades a tablas.	Permite con objetos Java y relaciones en lugar de manipular manualmente
MySQL 8	<code>mysql-connector-j</code> , <code>application.properties</code> , scripts SQL	Base de datos relacional principal.	Es una tecnología habitual en el entorno empresarial, adecuada para el análisis y hallazgo
H2	dependencia <code>h2</code> , <code>src/test/resources/application.properties</code>	Base de datos en memoria para pruebas.	Permite tests sin depender de MySQL
HTML5	todas las plantillas <code>templates</code>	Estructura semántica de la interfaz.	Es suficiente para una aplicación renderizada en el servidor

Tecnología	Dónde aparece	Qué hace	Por qué encaja en el proyecto
CSS3	<code>static/css/style.css</code> , <code>static/css/app.css</code>	Diseño visual, responsive, tarjetas, badges y mapas.	Mejora la comprensión del riesgo por usuarios técnicos
Bootstrap Icons	CDN en <code>fragments/layout.html</code>	Iconos visuales para navegación y acciones.	Da claridad visual sin un frontend pesado.
Chart.js	CDN en <code>dashboard.html</code>	Gráficos del dashboard.	Representa la distribución de riesgo, severidad, categorías OSINT y tendencias
Leaflet	WebJar <code>leaflet</code> y plantillas de geolocalización	Mapas interactivos.	Permite coordinar acciones usando OpenStreetMap sin crear un sistema propio.
OpenStreetMap	<code>tile.openstreetmap.org</code> en plantillas	Teselas de mapa.	Solución rápida y apropiada para visualizar ubicaciones y autorizaciones
JUnit	<code>spring-boot-starter-test</code> , tests	Framework de pruebas.	Valida correctamente los servicios controlados

Tecnología	Dónde aparece	Qué hace	Por qué encaja en el proyecto
			utilidades flujos cri
Mockito	<code>mockito-core</code> , Surefire <code>javaagent</code>	Soporte para dobles de prueba.	La configur como ag evita wa de Byte en Java modern
PowerShell	carpeta <code>scripts</code>	Automatización local en Windows.	El proyec pensado ejecutar desde W y estos ayudan puertos enlaces públicos

La elección general prioriza simplicidad, trazabilidad y facilidad de defensa. No se ha introducido React, microservicios, JWT, Docker o WebFlux porque el alcance del proyecto es una aplicación MVC académica y profesionalmente explicable.

7. Estructura del repositorio

La estructura real del repositorio muestra una aplicación Spring Boot estándar con documentación y scripts auxiliares:

Ruta	Contenido real	Exp
<code>src/main/java</code>	Código Java principal	Con de a paq
<code>src/main/java/com/pepitobuscaerror/controller</code>	<code>DashboardController</code> , <code>CompanyController</code> , <code>AnalysisController</code> , <code>OsintController</code> , <code>GeolocationController</code> , etc.	Ges form moc Thy
<code>src/main/java/com/pepitobuscaerror/service</code>	Servicios de empresas, análisis, dashboard, OSINT, geolocalización, enlaces públicos y cálculo de riesgo.	Aloj neg coo repe vista
<code>src/main/java/com/pepitobuscaerror/service/checks</code>	Comprobaciones pasivas como DNS, correo, TLS, cabeceras, CT, fingerprint y recursos well-known.	Imp mot pasi Se
<code>src/main/java/com/pepitobuscaerror/repository</code>	Interfaces <code>JpaRepository</code>	Abs a da
<code>src/main/java/com/pepitobuscaerror/model</code>	Entidades JPA y enums	Defi rela esta
<code>src/main/java/com/pepitobuscaerror/dto</code>	Formularios y resultados	Evit enti toda entr
<code>src/main/java/com/pepitobuscaerror/util</code>	<code>TargetNormalizer</code>	Nor don para emp

Ruta	Contenido real	Exp
<code>src/main/resources/templates</code>	Vistas Thymeleaf	Pág dasl emp indi recc OSI geo doc errc
<code>src/main/resources/static</code>	CSS, JavaScript e imagen SVG	Dis ma dasl mar
<code>src/main/resources/application.properties</code>	Configuración principal	Pue dat vari ent enla
<code>src/test/java</code>	Pruebas JUnit y MockMvc	Con con con serv utili
<code>src/test/resources/application.properties</code>	Configuración de tests	Usa mer aut dro
<code>docs</code>	Documentación técnica y SQL	Mo con OSI prue scrip mer

Ruta	Contenido real	Exp
<code>scripts</code>	Scripts PowerShell	Ayu pue en c prej GPS
<code>pom.xml</code>	Configuración Maven	Dep vers plug Boo
<code>HELP.md</code>	Ayuda generada por Spring Initializr	Refe de l Wet JPA

En el repositorio inspeccionado no aparece un `README.md` en la raíz. Sí aparece `HELP.md`, que cumple una función de referencia inicial generada por Spring Initializr, y varios documentos propios dentro de `docs/` que explican configuración, modelo de datos, OSINT, geolocalización, pruebas y roadmap.

8. pom.xml

El `pom.xml` define el proyecto Maven `com.pelitobuscaerror:pepito-busca-error:0.0.1-SNAPSHOT`. El parent utilizado es:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.14</version>
</parent>
```

La versión de Java se fija en Java 17:

```
<properties>
  <java.version>17</java.version>
</properties>
```

Las dependencias principales existen por una razón concreta:

Dependencia	Alcance	Motivo
<code>spring-boot-starter-data-jpa</code>	Principal	Repositorios JPA, Hibernate y persistencia relacional.
<code>spring-boot-starter-thymeleaf</code>	Principal	Renderizado de vistas HTML server-side.
<code>spring-boot-starter-validation</code>	Principal	Validaciones con anotaciones como <code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> y <code>@AssertTrue</code> .
<code>spring-boot-starter-web</code>	Principal	Spring MVC, Tomcat embebido, controladores y formularios.
<code>org.webjars.npm:leaflet:1.9.4</code>	Principal	Integrar Leaflet desde WebJars para mapas.
<code>mysql-connector-j</code>	Runtime	Conexión real con MySQL 8.
<code>h2</code>	Runtime	Base de datos en memoria usada por tests.
<code>spring-boot-starter-test</code>	Test	JUnit, AssertJ, Spring Test y MockMvc.
<code>mockito-core</code>	Test	Soporte explícito para Mockito.

El bloque de plugins contiene tres elementos importantes:

Plugin	Función
<code>maven-dependency-plugin</code>	Crea propiedades de dependencias durante <code>initialize</code> , usadas para ubicar el JAR de Mockito.
<code>maven-surefire-plugin</code>	Ejecuta tests y configura <code>argLine</code> con <code>-javaagent:\${org.mockito:mockito-core:jar}</code> .
<code>spring-boot-maven-plugin</code>	Permite empaquetar y ejecutar la aplicación Spring Boot.

La configuración de Surefire es relevante porque evita el auto-attach dinámico de Mockito/Byte Buddy en versiones modernas de Java:

```
<argLine>-Xshare:off -javaagent:${org.mockito:mockito-core:jar}</argLine>
```

Esto no cambia la aplicación en producción; solo afecta al proceso de pruebas. Desde un punto de vista de calidad, muestra que el proyecto no se limita a compilar, sino que también cuida la ejecución limpia de tests.

9. application.properties

La configuración principal se encuentra en

`src/main/resources/application.properties`. Está diseñada para funcionar en local, pero permitiendo sobrescribir valores mediante variables de entorno.

Fragmento representativo:

```
server.port=${SERVER_PORT:8080}
server.address=${SERVER_ADDRESS:0.0.0.0}
server.forward-headers-strategy=${SERVER_FORWARD_HEADERS_STRATEGY:framework}
app.public-base-url=${APP_PUBLIC_BASE_URL:}
public.base-url=${PUBLIC_BASE_URL:}

spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/pepito_busca_error?
createDatabaseIfNotExist=true&useSSL=false&serverTimezone=Europe/Madrid&allowPublicKeyRetrieval=true}
spring.datasource.username=${DB_USERNAME:pepito_app}
spring.datasource.password=${DB_PASSWORD:change_this_password}
spring.jpa.hibernate.ddl-auto=update

osint.securitytrails.api-key=${SECURITYTRAILS_API_KEY:}
osint.hibp.api-key=${HIBP_API_KEY:}
osint.demo-mode=${OSINT_DEMO_MODE:true}
```

Propiedad	Explicación
<code>SERVER_PORT</code>	Permite arrancar en otro puerto si <code>8080</code> está ocupado.
<code>SERVER_ADDRESS=0.0.0.0</code>	Hace que la aplicación escuche en todas las interfaces, útil para túneles o proxy inverso.
<code>APP_PUBLIC_BASE_URL</code>	URL pública HTTPS preferida para generar enlaces GPS que funcionen fuera de la red local.
<code>PUBLIC_BASE_URL</code>	Alias de compatibilidad para la URL pública.
<code>DB_URL</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code>	Permiten configurar MySQL sin cambiar código fuente.
<code>spring.jpa.hibernate.ddl-auto=update</code>	Hibernate actualiza el esquema en desarrollo.
<code>spring.jpa.open-in-view=false</code>	Evita mantener sesiones JPA abiertas durante la vista.
<code>osint.demo-mode=true</code>	Mantiene el módulo OSINT funcional sin claves externas.
<code>SECURITYTRAILS_API_KEY</code> , <code>HIBP_API_KEY</code>	Claves opcionales para proveedores externos.

La decisión de no hardcodear secretos es correcta: las claves API y credenciales reales no deben ir en Git. En desarrollo se usan valores por defecto seguros para arrancar, pero en producción se deberían inyectar variables de entorno reales, contraseñas fuertes y una URL pública HTTPS estable.

Para tests, `src/test/resources/application.properties` cambia MySQL por H2:

```
spring.datasource.url=jdbc:h2:mem:pepito_busca_error;MODE=MySQL;DATABASE_TO_LOWER=TRUE
spring.jpa.hibernate.ddl-auto=create-drop
```

Con esto, los tests no dependen de una base de datos local ni modifican datos reales.

10. Modelo de base de datos

El modelo relacional se basa en dos bloques: el bloque académico de compañías y análisis, y el bloque OSINT/geolocalización. Las tablas documentadas en `docs/database-model.md` y `docs/mysql-workbench-setup.sql` son:

Tabla	Propósito	Clave primaria
<code>companies</code>	Empresas registradas.	<code>id_company</code>
<code>analyses</code>	Evaluaciones de riesgo de una empresa.	<code>id_analysis</code>
<code>indicators</code>	Indicadores detectados en un análisis.	<code>id_indicator</code>
<code>recommendations</code>	Acciones recomendadas para un análisis.	<code>id_recommendation</code>
<code>audit_target</code>	Dominio o URL auditada por OSINT.	<code>id</code>
<code>scan_run</code>	Ejecución de informe OSINT pasivo.	<code>id</code>
<code>finding</code>	Hallazgo producido por un scan OSINT.	<code>id</code>
<code>tracked_devices</code>	Dispositivo propio o autorizado para geolocalización consentida.	<code>id_device</code>

Modelo relacional textual:

```
companies (1) — (N) analyses
analyses (1) — (N) indicators
analyses (1) — (N) recommendations

audit_target (1) — (N) scan_run
scan_run (1) — (N) finding

tracked_devices es independiente y guarda el último estado GPS autorizado.
```

companies

Guarda los datos básicos de la empresa: `name`, `domain`, `corporate_email`, `sector` y `registration_date`. Existe porque el resto del sistema necesita una entidad de negocio sobre la que crear análisis. Un ejemplo real de uso es registrar una pyme con dominio `mercadosol.es`, correo técnico y sector `Retail`.

analyses

Guarda cada evaluación de riesgo: fecha, puntuación, nivel, estado y empresa asociada mediante `id_company`. La relación con `Company` se implementa con `@ManyToOne` y `@JoinColumn(name = "id_company")`. Un análisis permite comparar evolución temporal de una empresa.

indicators

Guarda señales concretas de riesgo: tipo, valor, descripción, severidad y análisis asociado. Cada indicador pertenece a un análisis. En la base de datos `indicator_value` corresponde al atributo Java `value`.

recommendations

Guarda acciones recomendadas con prioridad, descripción y acción práctica. Cada recomendación pertenece a un análisis. Su valor de negocio es transformar un problema técnico en una tarea ejecutable.

audit_target

Guarda objetivos OSINT normalizados: nombre, dominio, URL y fecha de creación. Existe para no repetir el mismo dominio como texto suelto en cada informe.

scan_run

Representa una ejecución de comprobaciones pasivas. Guarda objetivo, fecha de inicio, fecha de finalización, estado y puntuación OSINT. La entidad `ScanRun` contiene una lista de `Finding` con `cascade = ALL` y `orphanRemoval = true`.

finding

Guarda un hallazgo OSINT con categoría, severidad, estado, título, evidencia y recomendación. Permite ordenar y mostrar hallazgos de DNS, correo, web, datos, OSINT y disponibilidad.

tracked_devices

Guarda dispositivos autorizados con nombre, tipo, propietario, token privado, estado activo, coordenadas opcionales, precisión, etiqueta de ubicación, IP, user-agent y fechas. No almacena histórico completo de ubicaciones; solo la última posición conocida.

Las claves foráneas principales son `analyses.id_company`, `indicators.id_analysis`, `recommendations.id_analysis`, `scan_run.target_id` y `finding.scan_run_id`. En las tablas de análisis, indicadores y recomendaciones se usa cascada para que al borrar una empresa se eliminen los datos dependientes. En OSINT, el borrado de informes y objetivos se gestiona desde `AuditService` y también existe cascada desde `ScanRun` hacia `Finding`.

11. Entidades Java

Las entidades se encuentran en `src/main/java/com/pepitobuscaerror/model`. Cada una representa una tabla o concepto persistente.

Entidad	Tabla	Propósito principal
Company	companies	Empresa registrada y resumen de su historial de análisis.
Analysis	analyses	Evaluación de riesgo de una empresa.
Indicator	indicators	Señal técnica detectada en un análisis.
Recommendation	recommendations	Acción recomendada asociada al análisis.
AuditTarget	audit_target	Objetivo normalizado de OSINT.
ScanRun	scan_run	Informe OSINT pasivo guardado.
Finding	finding	Hallazgo de un informe OSINT.
TrackedDevice	tracked_devices	Dispositivo geolocalizable con consentimiento.

Company

`Company` usa `@Entity` y `@Table(name = "companies")`. Sus atributos principales son `idCompany`, `name`, `domain`, `corporateEmail`, `sector`, `registrationDate` y `analyses`. Incluye validaciones como `@NotBlank`, `@Email` y `@Size`. La relación con análisis se define como:

```
@OneToMany(mappedBy = "company", cascade = CascadeType.ALL, orphanRemoval = true)
@OrderBy("analysisDate DESC")
private List<Analysis> analyses = new ArrayList<>();
```

Además de representar datos, contiene métodos útiles para la interfaz: último análisis, último nivel de riesgo, puntuación media, tendencia y número de recomendaciones. Esta decisión permite que la vista de detalle de empresa muestre un resumen sin duplicar esa lógica en la plantilla.

Analysis

Analysis representa un análisis terminado. Tiene **riskScore**, **riskLevel**, **status**, **analysisDate**, **company**, **indicators** y **recommendations**. El método **complete** limita la puntuación entre 0 y 100 y asigna el nivel:

```
public void complete(int riskScore, RiskLevel riskLevel) {  
    this.riskScore = Math.min(100, Math.max(0, riskScore));  
    this.riskLevel = riskLevel;  
    this.status = "COMPLETED";  
}
```

También incluye métodos **addIndicator** y **addRecommendation**, que adjuntan correctamente los objetos hijos al análisis.

Indicator

Indicator guarda **type**, **value**, **description**, **severity** y la relación **@ManyToOne** con **Analysis**. La severidad se persiste como texto mediante **@Enumerated(EnumType.STRING)**, lo que hace la base de datos más legible que si se guardaran ordinales.

Recommendation

Recommendation guarda **priority**, **description** y **action**. También calcula valores de presentación como responsable sugerido, plazo sugerido y estado de seguimiento. No sustituye a un gestor de tickets, pero permite mostrar un plan de acción estructurado.

AuditTarget

AuditTarget almacena nombre, dominio, URL normalizada y fecha de creación. La relación con **ScanRun** es uno a muchos. Esta entidad separa el concepto de empresa interna del concepto de activo público OSINT.

ScanRun

ScanRun representa una ejecución de informe. Comienza con estado **RUNNING**, puede terminar como **COMPLETED** o **FAILED**, y calcula etiquetas de riesgo para la interfaz. Los hallazgos se cargan eager para simplificar la vista del informe:

```
@OneToMany(mappedBy = "scanRun", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.EAGER)
@OrderBy("severity DESC, category ASC, id ASC")
private List<Finding> findings = new ArrayList<>();
```

Finding

Finding guarda la categoría, severidad, estado, título, evidencia y recomendación. El estado por defecto es **OPEN**. Esto deja preparada la base para una futura gestión de estados de hallazgos.

TrackedDevice

TrackedDevice guarda un dispositivo autorizado. Incluye validaciones de latitud y longitud, token privado, estado activo, metadata del cliente y fechas. El token se genera con **UUID.randomUUID()** si no existe. La entidad no intenta rastrear sin permiso: solo guarda posiciones enviadas desde la página live tras autorización del navegador.

Enums

Enum	Valores	Uso
RiskLevel	LOW , MEDIUM , HIGH , CRITICAL	Nivel del análisis de empresa.
Severity	LOW , MEDIUM , HIGH , CRITICAL	Severidad de indicadores.
FindingSeverity	INFO , LOW , MEDIUM , HIGH , CRITICAL	Severidad de hallazgos OSINT, con peso de puntuación.
FindingCategory	OSINT , WEB , MAIL , DNS , DATA , AVAILABILITY	Área técnica del hallazgo.
ScanStatus	RUNNING , COMPLETED , FAILED	Estado de informes OSINT.
Priority	LOW , MEDIUM , HIGH	Prioridad de recomendaciones.
FindingStatus	OPEN , IN_PROGRESS , ACCEPTED_RISK , RESOLVED , FALSE_POSITIVE	Estado futuro o actual de seguimiento de hallazgos.

12. Repositorios

Los repositorios usan Spring Data JPA y extienden `JpaRepository`. Esto proporciona automáticamente operaciones como `findAll`, `findById`, `save`, `deleteById`, `count` y `existsById`. Además, el proyecto define consultas por nombre de método y algunas consultas JPQL.

Repositorio	Entidad	Métodos destacables
<code>CompanyRepository</code>	<code>Company</code>	<code>findAllOrderByRegistrationDateDesc</code> , <code>findTop5ByOrderByRegistrationDateDesc</code> , búsqueda por nombre/dominio/sector.
<code>AnalysisRepository</code>	<code>Analysis</code>	Últimos análisis, conteos por <code>RiskLevel</code> , método de riesgo, conteo de indicadores por severidad.
<code>IndicatorRepository</code>	<code>Indicator</code>	Listado por severidad y tipo, conteo por <code>Severity</code> .
<code>RecommendationRepository</code>	<code>Recommendation</code>	Listado por prioridad, conteo por <code>Priority</code> .
<code>AuditTargetRepository</code>	<code>AuditTarget</code>	Búsqueda por dominio ignorando mayúsculas, últimos objetivos.
<code>ScanRunRepository</code>	<code>ScanRun</code>	Últimos scans, scans por target, conteo de hallazgos por severidad y categoría.
<code>TrackedDeviceRepository</code>	<code>TrackedDevice</code>	Búsqueda por token, búsquedas por texto, conteo de activos y localizados.

Se usan `@EntityGraph` en repositorios como `CompanyRepository`, `AnalysisRepository`, `IndicatorRepository` y `RecommendationRepository` para cargar relaciones necesarias en las vistas sin provocar problemas de carga perezosa. Esta decisión es coherente con `spring.jpa.open-in-view=false`, porque obliga a traer los datos que se van a usar antes de renderizar Thymeleaf.

Los repositorios mantienen la lógica de base de datos separada de los controladores. Por ejemplo, `CompanyController` no sabe cómo se ordenan las empresas ni cómo se busca por sector; delega en `CompanyService`, y este usa `CompanyRepository`.

13. Servicios

La capa de servicios contiene la lógica de negocio. Esta es una de las decisiones más importantes del proyecto, porque evita que los controladores se conviertan en clases con demasiada responsabilidad.

Servicio	Responsabilidad	Repositorios / dependencias	Co
CompanyService	Buscar, crear, actualizar y borrar compañías; normalizar dominio; inicializar datos de dashboard.	CompanyRepository , TargetNormalizer	C A
AnalysisService	Catálogo de indicadores, creación de análisis, cálculo de riesgo y recomendaciones.	CompanyRepository , AnalysisRepository , BasicRiskCalculator	A I
DashboardService	Construir KPIs, distribuciones y listas recientes.	CompanyRepository , AnalysisRepository , RecommendationRepository , ScanRunRepository	D
AuditService	Crear objetivos OSINT, ejecutar checks pasivos, guardar scans y ordenar hallazgos.	AuditTargetRepository , ScanRunRepository , lista de SecurityCheck	O
RiskCalculator	Contrato para calcular riesgo.	No aplica	Im B
BasicRiskCalculator	Ponderación de indicadores y umbrales de nivel de riesgo.	No aplica	A
IndicatorService	Listado de indicadores y catálogo disponible.	IndicatorRepository , AnalysisService	I
RecommendationService	Listado y conteo de recomendaciones por prioridad.	RecommendationRepository	R
OsintService	Coordinar DNSDumpster-style, SecurityTrails-style y HIBP-style; crear resultados y hallazgos.	DnsDumpsterClient , SecurityTrailsClient , HaveIBeenPwnedClient	O O

Servicio	Responsabilidad	Repositorios / dependencias	Código
<code>DnsLookupService</code>	Consultas DNS JNDI para A, AAAA, MX, NS, TXT, etc.	JNDI DNS	Clientes DNS
<code>DnsDumpsterClient</code>	Inteligencia pasiva estilo DnsDumpster con resumen DNS y datos demo.	<code>DnsLookupService</code> , <code>osint.demo-mode</code>	0
<code>SecurityTrailsClient</code>	Integración real o demo con SecurityTrails.	<code>RestTemplate</code> , <code>ObjectMapper</code> , API key opcional	0
<code>HaveIBeenPwnedClient</code>	Integración real o demo con Have I Been Pwned.	<code>RestTemplate</code> , <code>ObjectMapper</code> , API key opcional	0
<code>PublicLinkService</code>	Construir y clasificar enlaces GPS públicos, locales o LAN.	<code>Environment</code> , <code>HttpServletRequest</code>	G
<code>TrackingLinkService</code>	Servicio alternativo/compatibilidad para URLs de tracking.	<code>Environment</code>	Te: mo
<code>TrackedDeviceService</code>	CRUD de dispositivos y actualización de ubicación live validada.	<code>TrackedDeviceRepository</code>	G
<code>GeolocationSchemaService</code>	Ajuste de columnas GPS antiguas a nullable al arrancar.	<code>JdbcTemplate</code>	Ev: A

Checks pasivos de seguridad

El paquete `service.checks` implementa una arquitectura extensible mediante la interfaz:

```
public interface SecurityCheck {  
    List<Finding> analyze(AuditTarget target);  
}
```

Spring inyecta en `AuditService` una lista de implementaciones. El servicio las ordena por nombre de clase y las ejecuta. Las implementaciones reales son:

Check	Qué revisa	Enfoque
<code>DnsRecordCheck</code>	A/AAAA, NS, CAA, DS y CNAME obsoleto.	DNS pasivo.
<code>MailSecurityCheck</code>	MX, SPF, DMARC, MTA-STS y TLS-RPT.	Seguridad de correo.
<code>PassiveOsintFootprintCheck</code>	Inventario DNS y proveedores probables.	Huella pública.
<code>OsintProviderCheck</code>	Convierte resultados del módulo OSINT en findings guardados.	Integración de proveedores.
<code>WebSecurityHeaderCheck</code>	HTTPS, códigos HTTP, HSTS, CSP, cookies y disclosure.	Cabeceras web pasivas.
<code>WebFingerprintCheck</code>	Server, X-Powered-By, CDN/edge providers.	Fingerprinting no intrusivo.
<code>TlsCertificateCheck</code>	Handshake TLS, certificado, emisor y expiración.	Revisión TLS.
<code>CertificateTransparencyCheck</code>	Nombres en CT logs y etiquetas sensibles.	OSINT público.
<code>WellKnownResourceCheck</code>	<code>security.txt</code> , <code>robots.txt</code> , <code>sitemap.xml</code> .	Recursos públicos conocidos.
<code>HttpProbeClient</code>	Cliente HTTP común para checks web.	Infraestructura reutilizable.

La lógica pertenece a servicios porque es reutilizable, testeable y no depende directamente de una vista concreta.

14. Controladores

Los controladores se encuentran en `src/main/java/com/pepitobuscaerror/controller`. Su función es recibir peticiones, validar formularios, invocar servicios y devolver el nombre de la plantilla Thymeleaf.

Controlador	Rutas principales	Servicios
<code>DashboardController</code>	<code>GET /</code> , <code>GET /dashboard</code>	<code>DashboardServ</code>
<code>CompanyController</code>	<code>/companies</code> , <code>/companies/new</code> , <code>/companies/save</code> , <code>/companies/{id}</code> , edición y borrado	<code>CompanyServic</code>
<code>AnalysisController</code>	<code>/analyses</code> , <code>/analyses/new</code> , <code>/companies/{companyId}/analyses/new</code> , guardar y ver análisis	<code>CompanyServic</code> <code>AnalysisServi</code>
<code>IndicatorController</code>	<code>GET /indicators</code>	<code>IndicatorServ</code>
<code>RecommendationController</code>	<code>GET /recommendations</code>	<code>Recommendatio</code>
<code>OsintController</code>	<code>/osint</code> , <code>/osint/domain</code> , <code>/osint/email</code> , <code>/osint/run</code> , <code>/osint/scans/{id}</code>	<code>AuditService</code> , <code>OsintService</code>
<code>GeolocationController</code>	<code>/geolocation</code> , CRUD de dispositivos, <code>/live/{token}</code> , endpoints JSON de posición	<code>TrackedDevice</code> <code>PublicLinkSer</code>
<code>DocumentationController</code>	<code>GET /documentation</code>	Ninguno
<code>GlobalExceptionHandler</code>	Excepciones globales	<code>ResourceNotFo</code>

Ejemplo de flujo en creación de empresa:

1. El usuario abre `GET /companies/new`.

2. `CompanyController` coloca un objeto `Company` vacío en el modelo.
3. Thymeleaf renderiza `companies/form`.
4. El usuario envía `POST /companies/save`.
5. Spring valida `@Valid @ModelAttribute("company")`.
6. Si hay errores, se devuelve el formulario.
7. Si es válido, `CompanyService.createCompany` normaliza y guarda.
8. El controlador redirige al detalle de la empresa.

Ejemplo de endpoint JSON en geolocalización:

- `GET /geolocation/{id}/position` devuelve el estado actual del dispositivo.
- `POST /geolocation/live/{trackingToken}/position` recibe `latitude`, `longitude`, `accuracy` y `locationLabel` desde el navegador autorizado.

La aplicación mezcla vistas HTML y respuestas JSON solo donde tiene sentido: el panel es MVC/Thymeleaf, pero el mapa necesita refrescar posición sin recargar toda la página.

15. Dashboard y gráficos

El dashboard se apoya en `DashboardService` y `DashboardStats`. El servicio consulta repositorios y el DTO prepara datos ya listos para Thymeleaf y Chart.js.

La página `dashboard.html` muestra:

- Tarjetas KPI de empresas, análisis, OSINT checks, riesgo medio, hallazgos críticos, hallazgos altos y recomendaciones abiertas.
- Visualización animada de exposición en `dashboard-exposure-map.js`.
- Gráfico de análisis por nivel de riesgo.
- Gráfico de indicadores por severidad.
- Gráfico de hallazgos OSINT por área.
- Tendencia de puntuaciones recientes.
- Últimas empresas registradas.
- Últimos scans OSINT.
- Top recomendaciones.
- Bloque de análisis críticos y altos.

Datos reales que llegan al frontend:

Dato	Origen backend
Total de empresas	<code>companyRepository.count()</code>
Total de análisis	<code>analysisRepository.count()</code>
Riesgo medio	<code>analysisRepository.averageRiskScore()</code>
Análisis críticos	<code>countByRiskLevel(RiskLevel.CRITICAL)</code>
Indicadores críticos	<code>countIndicatorsBySeverity(Severity.CRITICAL)</code>
Hallazgos críticos OSINT	<code>scanRunRepository.countFindingsBySeverity(FindingSeverity.CRITICAL)</code>
Scans recientes	<code>findTop12ByOrderByStartedAtDesc()</code>
Tendencia reciente	<code>latestAnalyses</code> ordenados por fecha

El paso desde backend a frontend es directo: `DashboardController` añade `stats` al modelo y Thymeleaf usa expresiones como `${stats.totalCompanies}`, `${stats.riskDistributionData}` o `${stats.recentRiskTrendLabels}`. Chart.js se inicializa en un script inline de `dashboard.html`.

El proyecto cuida los estados vacíos. Si no hay datos, la vista no dibuja gráficos falsos: muestra mensajes de estado vacío indicando que primero hay que crear análisis o informes. Esto es importante en una presentación porque permite demostrar honestidad de datos y evitar una interfaz engañosa.

16. Módulo de empresas

El módulo de empresas es la base del sistema. Sin una empresa registrada, no tiene sentido crear análisis de riesgo.

Funcionalidades reales:

- Listar empresas en `/companies`.
- Buscar por nombre, dominio o sector mediante el parámetro `q`.
- Crear empresa en `/companies/new` y `POST /companies/save`.
- Ver detalle en `/companies/{id}`.
- Editar empresa en `/companies/edit/{id}` y `POST /companies/update/{id}`.
- Borrar empresa con `/companies/delete/{id}`.
- Lanzar análisis desde el detalle de empresa.
- Abrir OSINT pre-rellenando dominio y nombre.

Campos principales:

Campo	Uso
<code>name</code>	Nombre de la compañía.
<code>domain</code>	Dominio público que se analizará o usará para OSINT.
<code>corporateEmail</code>	Correo técnico/corporativo.
<code>sector</code>	Sector de actividad.
<code>registrationDate</code>	Fecha automática de registro.

`CompanyService` normaliza el dominio con `TargetNormalizer`, recorta espacios y guarda la entidad. La relación `Company → Analysis` permite mostrar en el detalle la puntuación más reciente, media histórica, tendencia y número de hallazgos/recomendaciones. Este diseño convierte la empresa en el punto de entrada natural para el resto del sistema.

17. Módulo de análisis de riesgo

El módulo de análisis de riesgo permite crear una evaluación seleccionando indicadores predefinidos. No inventa hallazgos automáticos: en esta parte el usuario selecciona indicadores del catálogo definido en `AnalysisService.availableIndicatorOptions()`.

Flujo real:

1. Se elige una empresa.
2. Se abre `/companies/{companyId}/analyses/new`.
3. El controlador carga `company`, `analysisForm` e `indicatorOptions`.
4. El usuario selecciona indicadores.
5. `AnalysisService.createAnalysis` construye objetos `Indicator`.
6. `BasicRiskCalculator` calcula la puntuación.
7. Se asigna `RiskLevel`.
8. Se generan recomendaciones según los indicadores.
9. Se guarda `Analysis` con sus indicadores y recomendaciones.
10. Se muestra `analyses/result` agrupando indicadores por severidad y recomendaciones por prioridad.

Catálogo real de indicadores:

Indicador	Severidad	Puntuación
Suspicious IP address	CRITICAL	40
Suspicious corporate email	HIGH	25
Domain without HTTPS	MEDIUM	15
Missing security headers	MEDIUM	10
Invalid SSL certificate	HIGH	10

Ejemplo pedido: una empresa tiene tres indicadores:

- Missing HTTPS, que en el código corresponde a `Domain without HTTPS` : 15 puntos.
- Missing security headers: 10 puntos.
- Exposed email breach, que en el módulo de análisis se representa como `Suspicious corporate email` : 25 puntos.

Cálculo según el código:

```
15 + 10 + 25 = 50 puntos
```

Umbral reales de `BasicRiskCalculator` :

```
0-30    → LOW  
31-60   → MEDIUM  
61-80   → HIGH  
81-100  → CRITICAL
```

Por tanto, el resultado sería `50` , nivel `MEDIUM` . Las recomendaciones generadas serían:

- `Enable HTTPS.` con acción de instalar certificado TLS válido y redirigir HTTP a HTTPS.
- `Review security headers.` con acción de añadir HSTS, CSP, X-Content-Type-Options, Referrer-Policy y cabeceras relacionadas.
- `Change exposed credentials.` con acción de resetear credenciales sospechosas y revisar SPF, DKIM y DMARC.

Este ejemplo demuestra que el motor es simple, transparente y defendible: cualquier puntuación puede explicarse leyendo las reglas.

18. Motor de riesgo

El motor de riesgo está separado mediante la interfaz `RiskCalculator` y la implementación `BasicRiskCalculator`. La interfaz define el contrato:

```
public interface RiskCalculator {  
    int calculateRisk(List<Indicator> indicators);  
}
```

`BasicRiskCalculator` suma pesos por tipo de indicador y limita el resultado a 100. Después asigna el nivel:

```
if (riskScore ≤ 30) return RiskLevel.LOW;  
if (riskScore ≤ 60) return RiskLevel.MEDIUM;  
if (riskScore ≤ 80) return RiskLevel.HIGH;  
return RiskLevel.CRITICAL;
```

La ventaja de este sistema basado en reglas es que es fácil de explicar, testear y modificar. Para un proyecto DAM es más adecuado que un sistema de machine learning porque no exige datasets, entrenamiento ni opacidad algorítmica. Además, los tests `BasicRiskCalculatorTests` comprueban puntuaciones conocidas, límite máximo y umbrales.

Su principal limitación es que los pesos son estáticos. En una versión futura se podrían parametrizar por sector, histórico, criticidad del activo o frecuencia de aparición. Aun así, para el alcance actual, el diseño cumple su función: convertir indicadores técnicos en una puntuación entendible.

19. Indicadores

Los indicadores representan señales concretas que afectan al riesgo de una empresa. La entidad `Indicator` tiene tipo, valor, descripción, severidad y referencia al análisis.

Campos relevantes:

Campo	Explicación
<code>type</code>	Nombre técnico del indicador.
<code>value</code>	Valor resumido o evidencia breve.
<code>description</code>	Explicación del problema.
<code>severity</code>	Nivel de severidad: LOW, MEDIUM, HIGH o CRITICAL.
<code>analysis</code>	Análisis al que pertenece.

El módulo de indicadores tiene dos usos:

- Mostrar indicadores ya almacenados en `/indicators`.
- Mostrar el catálogo disponible de checks manuales definido por `AnalysisService`.

Los indicadores alimentan el dashboard mediante conteos por severidad y alimentan recomendaciones porque cada indicador puede activar una acción. Por ejemplo, `Domain without HTTPS` activa `Enable HTTPS`, mientras que `Missing security headers` activa `Review security headers`.

20. Recomendaciones y plan de acción

Las recomendaciones convierten problemas en acciones. La entidad `Recommendation` guarda prioridad, descripción y acción. También proporciona métodos de presentación como responsable sugerido y plazo sugerido:

Prioridad	Responsable sugerido	Plazo sugerido
HIGH	IT / Security owner	7 days
MEDIUM	Technical owner	30 days
LOW	Operations owner	90 days

La generación se realiza en `AnalysisService.generateRecommendations`. El uso de un `LinkedHashMap` evita duplicar recomendaciones con la misma descripción.

Ejemplos reales:

Indicador	Recomendación	Acción
Suspicious IP address	Review IP reputation.	Revisar IP, proveedor, DNS y exposición firewall.
Suspicious corporate email	Change exposed credentials.	Resetear credenciales y revisar SPF, DKIM y DMARC.
Domain without HTTPS	Enable HTTPS.	Instalar TLS válido y redirigir HTTP a HTTPS.
Missing security headers	Review security headers.	Añadir HSTS, CSP, X-Content-Type-Options, Referrer-Policy, etc.
Invalid SSL certificate	Renew or install SSL certificate.	Sustituir certificados expirados o inválidos.

Desde el punto de vista empresarial, este módulo es clave porque una pyme no solo necesita saber que tiene riesgo: necesita saber qué hacer primero.

21. Módulo OSINT

El módulo OSINT tiene un enfoque pasivo, defensivo y autorizado. Está documentado en `docs/osint-module.md` y se implementa principalmente con `OsintController`, `OsintService`, clientes de proveedor y `AuditService`.

Rutas reales:

Ruta	Función
<code>GET /osint</code>	Página principal con formularios de dominio, correo y scan guardado.
<code>POST /osint/domain</code>	Análisis de dominio estilo DNSDumpster/SecurityTrails.
<code>POST /osint/email</code>	Comprobación de correo estilo Have I Been Pwned.
<code>POST /osint/run</code>	Ejecuta y guarda un informe OSINT pasivo completo.
<code>GET /osint/scans/{id}</code>	Muestra detalle de informe guardado.
<code>POST /osint/scans/{id}/delete</code>	Borra un informe.
<code>POST /osint/targets/{id}/delete</code>	Borra un objetivo y sus informes.

Los formularios `OsintDomainForm`, `OsintEmailForm` y `TargetForm` obligan a marcar autorización mediante `@AssertTrue`. Esto es una decisión ética importante: la interfaz recuerda que solo se deben analizar dominios, correos o activos propios o autorizados.

El módulo analiza:

- Registros DNS A, AAAA, MX, NS, TXT, CAA y DS.
- Subdominios demo o de proveedor.
- Proveedores asociados por registros públicos.
- Seguridad de correo: SPF, DMARC, MTA-STS y TLS-RPT.

- Cabeceras web y cookies.
- Metadatos TLS y expiración de certificados.
- Certificate Transparency logs.
- `security.txt`, `robots.txt` y `sitemap.xml`.
- Exposición de correo corporativo mediante resultados tipo HIBP.

El modo demo existe para que el proyecto funcione sin claves API de pago. Si no hay claves o `OSINT_DEMO_MODE=true`, los clientes devuelven datos de demostración claramente indicados. Esto evita que la aplicación falle durante una presentación académica y evita hardcodear secretos.

Los hallazgos se almacenan como `Finding` dentro de `ScanRun` cuando se ejecuta el flujo guardado. En el análisis directo de dominio o email se muestran DTOs de resultado sin necesariamente persistirlos.

22. APIs externas

El proyecto integra APIs o integraciones estilo API de forma opcional y segura.

Integración	Clase	Modo real
DNS lookups	<code>DnsLookupService</code>	JNDI DNS local para registros públicos.
DNSDumpster-style	<code>DnsDumpsterClient</code>	Resume DNS real y genera subdominios no intrusivos simulados.
SecurityTrails	<code>SecurityTrailsClient</code>	Llama a <code>https://api.securitytrails.com/v1/domain/</code> con header <code>APIKEY</code> .
Have I Been Pwned	<code>HaveIBeenPwnedClient</code>	Llama a breachedaccount API con <code>hibp-api-key</code> y user-agent.
HTTP/TLS públicos	<code>HttpProbeClient</code> , <code>TlsCertificateCheck</code>	Solicitudes HTTP GET y handshake TLS.
OpenStreetMap	Plantillas geolocalización	Carga teselas públicas.

Las claves no están hardcodeadas. Se leen desde propiedades:

```
osint.securitytrails.api-key=${SECURITYTRAILS_API_KEY:}  
osint.hibp.api-key=${HIBP_API_KEY:}  
osint.demo-mode=${OSINT_DEMO_MODE:true}
```

Las limitaciones son claras: los datos de proveedores externos pueden tener límites de cuota, coste, disponibilidad o exactitud. Por eso el código tiene fallback a demo y mensajes de estado. En privacidad, la aplicación solo muestra información defensiva de alto nivel; no solicita contraseñas ni intenta acceder a cuentas.

23. Módulo de geolocalización

El módulo de geolocalización está diseñado para dispositivos propios o explícitamente autorizados. No intenta ocultarse, no instala nada en el dispositivo y no evita los permisos del navegador.

Flujo real:

1. Se registra un dispositivo en `/geolocation/new`.
2. `TrackedDevice` genera un `trackingToken` privado.
3. En el detalle, `PublicLinkService` construye un enlace `/geolocation/live/{token}`.
4. El usuario abre ese enlace en el dispositivo autorizado.
5. El navegador muestra su permiso de geolocalización.
6. Solo si el usuario acepta, JavaScript envía latitud, longitud, precisión y etiqueta aproximada.
7. `GeolocationController` recibe la posición por POST.
8. `TrackedDeviceService` valida coordenadas y actualiza la última posición.
9. La página de detalle consulta `/geolocation/{id}/position` para refrescar el mapa.

Datos que puede guardar `TrackedDevice` :

Campo	Explicación
<code>trackingToken</code>	Token privado para el enlace live.
<code>latitude</code> , <code>longitude</code>	Últimas coordenadas autorizadas.
<code>accuracyMeters</code>	Precisión indicada por el navegador.
<code>locationLabel</code>	Etiqueta legible opcional.
<code>lastClientIp</code>	IP de la petición autorizada, considerando cabeceras de proxy.
<code>lastUserAgent</code>	Navegador del cliente que envió la posición.
<code>active</code>	Permite desactivar el tracking.
<code>registeredAt</code> , <code>lastSeenAt</code>	Fechas de registro y última actualización.

Leaflet se usa para el mapa interactivo y OpenStreetMap para las teselas. La plantilla `geolocation/detail.html` muestra la última ubicación, precisión, enlace público, estado del token y controles de mapa. La plantilla `geolocation/live.html` es mínima: solo solicita permiso y envía posición.

Punto de privacidad fundamental: la aplicación no bypassa permisos del navegador y no hace tracking oculto. Si la pestaña se cierra, deja de enviar actualizaciones. Para tracking continuo en segundo plano haría falta una app nativa o una PWA avanzada, que no forma parte del proyecto actual.

24. Public HTTPS links

El módulo GPS necesita enlaces que puedan abrirse desde otro dispositivo. Un enlace `localhost` solo funciona en el mismo ordenador. Un enlace `192.168.x.x` suele funcionar solo dentro de la misma red Wi-Fi. Para un móvil en datos o en otra red se necesita una URL pública HTTPS.

La configuración relevante es:

```
app.public-base-url=${APP_PUBLIC_BASE_URL:}
public.base-url=${PUBLIC_BASE_URL:}
server.forward-headers-strategy=${SERVER_FORWARD_HEADERS_STRATEGY:framework}
```

`PublicLinkService` construye el enlace con esta prioridad:

1. `app.public-base-url` / `APP_PUBLIC_BASE_URL`.
2. `public.base-url` / `PUBLIC_BASE_URL` como alias.
3. URL derivada de la petición actual, incluyendo cabeceras `Forwarded` y `X-Forwarded-*`.

Después clasifica el enlace:

Estado	Significado
<code>Public HTTPS ready</code>	Enlace público HTTPS válido para otra red.
<code>Local test link</code>	Solo sirve en el mismo ordenador.
<code>Same Wi-Fi only</code>	Puede servir en LAN, no desde otra red.
<code>HTTPS required</code>	Hay host público pero usa HTTP; la geolocalización móvil puede fallar.
<code>Not properly configured</code>	No se pudo generar una URL útil.

Esto es importante porque los navegadores móviles suelen exigir HTTPS para geolocalización precisa. Los documentos `docs/public-gps-link.md` y `scripts/README-public-`

[link.md](#) explican flujos con Cloudflare Tunnel o ngrok.

Ejemplo PowerShell documentado:

```
$env:APP_PUBLIC_BASE_URL="https://abc123.trycloudflare.com"  
.\mvnw.cmd spring-boot:run
```

La aplicación no puede convertir un [localhost](#) privado en público por sí sola. Necesita despliegue público, proxy inverso o túnel HTTPS.

25. Scripts PowerShell

La carpeta `scripts` contiene utilidades para desarrollo local en Windows.

Script	Propósito	Cuándo se usa	Problema que resuelve
<code>easy-public-gps-link.ps1</code>	Construye el JAR, arranca túnel Cloudflare/ngrok, configura <code>APP_PUBLIC_BASE_URL</code> y ejecuta Spring Boot.	Cuando se quiere probar GPS desde otra red con un solo flujo.	Evita copiar manualmente URLs de túnel y coordinar puerto/app.
<code>start-with-public-url.ps1</code>	Pide o recibe una URL pública HTTPS y arranca Maven con <code>APP_PUBLIC_BASE_URL</code> .	Cuando ya se tiene el túnel creado.	Simplifica establecer variables de entorno en PowerShell.
<code>run-on-port.ps1</code>	Establece <code>SERVER_PORT</code> y arranca <code>mvnw.cmd spring-boot:run</code> .	Cuando el puerto 8080 está ocupado o se quiere usar otro.	Evita editar <code>application.properties</code> .
<code>free-port.ps1</code>	Inspecciona qué proceso usa un puerto y pregunta si debe detenerlo.	Cuando Spring Boot no puede arrancar por puerto ocupado.	Ayuda a liberar puertos de forma explícita y con confirmación.
<code>README-public-link.md</code>	Guía de uso de enlaces GPS públicos.	Consulta del estudiante o del tribunal.	Documenta Cloudflare, ngrok, puertos y pruebas con móvil.

`easy-public-gps-link.ps1` es el script más avanzado. Primero resuelve el puerto, busca `cloudflared` o `ngrok`, construye el JAR con Maven, inicia el túnel, extrae la URL HTTPS,

configura variables y arranca la aplicación. También deja claro que el móvil debe pulsar **Allow** en el permiso del navegador.

Estos scripts son útiles porque el proyecto se encuentra en un entorno Windows (**mvnw.cmd** , PowerShell y ruta de escritorio). Automatizan tareas repetitivas sin modificar el código Java.

26. Templates Thymeleaf

Las plantillas están en `src/main/resources/templates`. El proyecto usa fragmentos reutilizables para evitar duplicar HTML.

Plantilla / carpeta	Función
<code>fragments/layout.html</code>	<code><head></code> común: título, Bootstrap Icons, Leaflet CSS y <code>style.css</code> .
<code>fragments/sidebar.html</code>	Navegación lateral principal.
<code>fragments/navbar.html</code>	Barra superior responsive.
<code>fragments/footer.html</code>	Pie con referencia a DAM, Spring Boot, Thymeleaf y MySQL.
<code>index.html</code>	Página inicial con resumen de riesgo.
<code>dashboard.html</code>	Dashboard completo con KPIs y Chart.js.
<code>companies/list.html</code>	Listado y búsqueda de empresas.
<code>companies/form.html</code>	Alta y edición de empresa.
<code>companies/detail.html</code>	Detalle, historial, últimos indicadores y recomendaciones.
<code>analyses/list.html</code>	Listado de análisis.
<code>analyses/form.html</code>	Selección de indicadores.
<code>analyses/result.html</code>	Resultado del análisis agrupado por severidad/prioridad.
<code>indicators/list.html</code>	Indicadores guardados y catálogo disponible.
<code>recommendations/list.html</code>	Plan de acciones por prioridad.
<code>osint/index.html</code>	Formularios OSINT y listas de objetivos/informes recientes.
<code>osint/domain-result.html</code>	Resultado de dominio con DNSDumpster/SecurityTrails-style.
<code>osint/email-result.html</code>	Resultado de email estilo HIBP.

Plantilla / carpeta	Función
<code>osint/detail.html</code>	Informe OSINT guardado con findings.
<code>scan-detail.html</code>	Plantilla de compatibilidad para informe OSINT.
<code>geolocation/list.html</code>	Listado de dispositivos.
<code>geolocation/form.html</code>	Alta/edición de dispositivo.
<code>geolocation/detail.html</code>	Mapa, enlace público y estado de ubicación.
<code>geolocation/live.html</code>	Página cliente que solicita permiso GPS.
<code>documentation/index.html</code>	Documentación interna resumida.
<code>error.html</code> , <code>error/404.html</code> , <code>error/500.html</code>	Páginas de error amigables.

Thymeleaf usa variables añadidas por los controladores. Ejemplos:

```
<strong th:text="${stats.totalCompanies}">0</strong>
<a th:href="@{/companies/{id}(id=${company.idCompany})}">
<span th:text="${analysis.riskLevel.label}">Low</span>
```

Esta elección permite que el HTML sea legible y que la lógica fuerte permanezca en Java. Las plantillas se encargan de presentar datos, no de calcular reglas de negocio.

27. CSS y diseño visual

El diseño principal está en `src/main/resources/static/css/style.css`. `app.css` importa `style.css` y añade compatibilidad para una página standalone de informe.

Elementos visuales importantes:

- Paleta basada en fondo claro, superficies blancas, azul de acento, verde, amarillo y rojo para riesgo.
- Layout tipo SaaS con sidebar, topbar, cards, métricas y tablas.
- Badges para riesgo, severidad, prioridad y estados.
- Tarjetas KPI con iconografía de Bootstrap Icons.
- Grids responsive para dashboard, detalles y formularios.
- Contenedores de charts con estados vacíos.
- Estilos específicos para mapas Leaflet y geolocalización.
- Media queries para adaptar el diseño a móvil.

La decisión visual tiene una justificación de negocio: los usuarios no técnicos entienden mejor el riesgo si se presenta con colores, porcentajes, tendencias y prioridades. Una tabla cruda de hallazgos puede ser correcta técnicamente, pero una pyme necesita saber qué es crítico, qué es alto y qué acciones debe priorizar.

El diseño evita depender de un framework frontend pesado. Se logra una apariencia profesional con CSS propio, Thymeleaf, Bootstrap Icons, Chart.js y Leaflet.

28. Error handling

La gestión de errores se centraliza en `GlobalExceptionHandler`, marcado con `@ControllerAdvice`. Esto evita mostrar trazas internas al usuario.

Casos gestionados:

Excepción	Respuesta	Vista
<code>ResourceNotFoundException</code>	404	<code>error/404</code>
<code>NoHandlerFoundException</code> / <code>NoResourceFoundException</code>	404	<code>error/404</code>
<code>MethodArgumentTypeMismatchException</code>	404	<code>error/404</code>
<code>Exception</code> genérica	500	<code>error/500</code>

El mensaje de 500 es deliberadamente genérico:

```
The application could not complete the requested action. Please return to the dashboard and try again.
```

Esto es correcto porque una aplicación no debe enseñar stack traces, nombres de clases internas, consultas SQL o detalles de infraestructura al usuario final. En producción, esos detalles deberían quedar en logs internos y observabilidad, no en HTML público.

29. Tests

El proyecto incluye pruebas en `src/test/java` y configuración H2 en `src/test/resources/application.properties`.

Tecnologías de test:

- JUnit 5.
- Spring Boot Test.
- MockMvc.
- H2 en memoria en modo MySQL.
- AssertJ.
- Mockito configurado como agente de test.

Cobertura real observada:

Test	Qué valida
<code>PepitoBuscaErrorApplicationTests</code>	Carga del contexto Spring.
<code>CorePageSmokeTests</code>	Renderizado de <code>/</code> , <code>/dashboard</code> , <code>/companies</code> , <code>/geolocation</code> , detalle de empresa, resultado de análisis y páginas live.
<code>OsintControllerTests</code>	Página OSINT, dominio demo, email demo, validación, detalle y borrados.
<code>GeolocationControllerTests</code>	Actualización live, rechazo de dispositivos inactivos y coordenadas inválidas.
<code>BasicRiskCalculatorTests</code>	Puntuaciones conocidas, límite a 100 y umbrales de nivel.
<code>CompanyRiskHistoryTests</code>	Media, tendencia y análisis previo.
<code>TargetNormalizerTests</code>	Normalización, puertos, dominios públicos y esquemas inválidos.
<code>PublicLinkServiceTests</code>	Clasificación de enlaces públicos, localhost, LAN, HTTP y alias.
<code>TrackingLinkServiceTests</code>	Elección de URL pública recomendada.
<code>HaveIBeenPwnedClientTests</code>	Demo mode cuando falta API key.
<code>HttpProbeClientTests</code>	Peticiones HTTP locales y captura de metadata.

La base H2 permite ejecutar pruebas sin MySQL local. El documento `docs/testing.md` también propone una prueba manual de humo: abrir dashboard, registrar empresa, crear análisis, revisar gráficos, ejecutar OSINT demo, probar geolocalización y confirmar errores 404 amigables.

Mejoras futuras razonables serían más tests de repositorio, más MockMvc para flujos de empresa, tests de integración de `AuditService` y pruebas visuales básicas de plantillas críticas.

30. Seguridad y privacidad

La aplicación está diseñada con propósito defensivo. No incluye explotación ofensiva, fuerza bruta, escaneo de redes, bypass de autenticación ni recolección de credenciales.

Medidas y decisiones reales:

- Formularios OSINT requieren confirmar autorización con `@AssertTrue`.
- `TargetNormalizer` rechaza localhost, IPs y dominios `.local` para OSINT.
- Las claves API se leen de variables de entorno.
- El modo demo evita hardcodear claves y mantiene la aplicación funcional.
- HIBP-style no solicita ni almacena contraseñas.
- Geolocalización depende del permiso del navegador.
- Los dispositivos pueden desactivarse con `active=false`.
- Coordenadas se validan en backend.
- La app guarda solo la última ubicación del dispositivo, no un histórico completo.
- Los errores no muestran trazas internas al usuario.

Limitación de seguridad actual importante: no hay autenticación ni roles implementados. Esto está documentado en `docs/security-roadmap.md` como decisión de alcance académico. En producción sería obligatorio añadir Spring Security, login, roles, CSRF, control de propiedad de datos, auditoría y políticas de retención.

También debe quedar claro que un enlace GPS público es un token privado. No debe publicarse en capturas, chats o repositorios. La seguridad del enlace depende de su confidencialidad y de que el dispositivo lo abra voluntariamente.

31. Uso de IA durante el desarrollo

Durante el desarrollo utilicé inteligencia artificial como herramienta de apoyo, no como sustituto de mi trabajo ni como servicio necesario para ejecutar la aplicación.

El uso de IA se puede explicar de forma honesta así:

- Me ayudó a planificar la arquitectura MVC y a ordenar ideas.
- Me sirvió para proponer estructuras de documentación técnica.
- La usé para revisar explicaciones y mejorar la redacción de textos de interfaz.
- Me ayudó a depurar errores concretos y a pensar casos de prueba.
- Me sirvió para preparar prompts, listas de comprobación y secciones de memoria.

No significa que la IA haya construido el proyecto de forma autónoma. Las decisiones finales, la adaptación al código real, la ejecución de pruebas, la integración de clases, la configuración local y la comprensión necesaria para defender el proyecto corresponden al estudiante.

Además, la aplicación no requiere ninguna API de IA para funcionar. Es una aplicación Java Spring Boot tradicional con MySQL, Thymeleaf, JPA y lógica local.

32. Limitaciones actuales

El proyecto es funcional como prototipo académico, pero tiene limitaciones que conviene reconocer:

- No hay autenticación ni roles de usuario implementados.
- No existe multi-tenant ni separación por organizaciones.
- No hay despliegue productivo completo documentado como infraestructura final.
- Las integraciones SecurityTrails y HIBP pueden funcionar en modo demo si no hay claves.
- El GPS desde otra red requiere HTTPS público mediante despliegue, proxy o túnel.
- La geolocalización solo funciona mientras la página live está abierta.
- No hay tracking histórico completo de posiciones.
- No hay scans programados automáticos.
- No hay sistema real de alertas por correo o webhook.
- No hay exportación PDF interna de informes de la aplicación como funcionalidad de producto.
- La puntuación de riesgo de análisis de empresa es rule-based y estática.
- La cobertura de tests es útil, pero no exhaustiva.

Estas limitaciones no invalidan el proyecto; al contrario, ayudan a acotar qué es versión académica y qué sería una versión empresarial endurecida.

33. Mejoras futuras

Las mejoras futuras documentadas y coherentes con el código actual son:

- Añadir Spring Security con login, logout y sesiones.
- Crear roles `ADMIN`, `ANALYST` y `VIEWER`.
- Añadir hash de contraseñas con BCrypt.
- Implementar organizaciones/multi-tenant para separar datos por cliente.
- Activar CSRF y reglas de acceso por rutas.
- Añadir informes PDF ejecutivos y técnicos desde la propia aplicación.
- Exportar CSV de hallazgos y recomendaciones.
- Añadir scans OSINT programados.
- Implementar alertas por email o webhook.
- Añadir más APIs OSINT con controles de privacidad y cuota.
- Permitir habilitar/deshabilitar proveedores OSINT.
- Gestionar estados de findings desde UI.
- Añadir expiración y regeneración de tokens GPS.
- Definir política de retención de ubicaciones y evidencias OSINT.
- Aumentar tests unitarios, tests MockMvc y tests de repositorio.
- Crear guía de despliegue productivo con HTTPS, variables de entorno, logs y backup.

Estas mejoras mantienen la dirección del proyecto: una plataforma defensiva para pymes, cada vez más robusta y profesional.

34. Conclusión

PepitoBuscaError es un proyecto defendible porque combina una arquitectura Spring Boot MVC clara con una temática profesional actual: la exposición digital y la ciberseguridad defensiva en pequeñas y medianas empresas.

Desde el punto de vista técnico, el proyecto demuestra dominio de Java 17, Spring Boot, controladores, servicios, repositorios, entidades JPA, validación, DTOs, Thymeleaf, MySQL, pruebas y scripts de automatización. La separación por capas permite explicar el código de forma ordenada y justificar por qué cada responsabilidad está donde corresponde.

Desde el punto de vista académico, la aplicación es adecuada para un proyecto final de DAM porque no es solo una colección de pantallas: tiene flujo de negocio, persistencia, cálculo de riesgo, visualización, integración opcional con APIs, privacidad, documentación, pruebas y mejoras futuras. También muestra criterio al no incorporar tecnologías innecesarias para el alcance.

Desde el punto de vista de negocio, la plataforma aporta valor porque convierte señales técnicas en información accionable. Una pyme puede registrar empresas, ver su riesgo, entender indicadores, priorizar recomendaciones, revisar huella pública y gestionar dispositivos autorizados con enlaces GPS seguros.

Lo más importante que he aprendido con este proyecto es a construir una aplicación completa manteniendo coherencia entre arquitectura, base de datos, interfaz, seguridad, privacidad y documentación. La aplicación no pretende ser una herramienta ofensiva ni una plataforma empresarial terminada, sino un prototipo sólido, explicable y extensible que demuestra una base profesional real.